

FINAL PROJECT REPORT

CyberGuide AI

Vibe Coding: Building and Deploying an AI Web Application on AWS

AI • SECURITY AWARENESS • CLOUD

Prepared by: [Student Name]

Course: Gen AI & Cloud Computing

Date: July 2026

LIVE AWS APPLICATION URL: [ADD AFTER DEPLOYMENT]

Executive Summary

Project outcome: CyberGuide AI is a full-stack security-awareness application with a responsive frontend, FastAPI backend, streamed Groq LLM integration, Docker packaging, and an AWS-ready deployment design.

The project demonstrates an AI-first development workflow in which the learner acts as architect, designer, tester, and prompt director. AI assistance was used to explore the concept, refine the interface, draft code, identify errors, and improve documentation. Final decisions about scope, safety, architecture, and user experience were retained by the learner.

1. Application Overview

CyberGuide AI helps users learn about digital threats and protective measures through an interactive AI assistant. Instead of presenting only an open chat box, the application organizes learning into seven visible domains. Users can select their level and choose whether they want an explanation, defensive measures, or a quiz.

Item	Details
Frontend	Semantic HTML, responsive CSS, and vanilla JavaScript
Backend	Python 3.12, FastAPI, Pydantic validation, and HTTPX
AI model	Llama 3.3 70B Versatile through the Groq API
Streaming	FastAPI StreamingResponse and browser ReadableStream
Container	Docker using a minimal Python base image and non-root user
Cloud	Amazon ECR image repository and AWS App Runner service

2. Problem and Design Rationale

Many users know that online threats exist but struggle to distinguish phishing, malware, social engineering, network risks, and cloud-security responsibilities. The application therefore combines topic navigation with conversational AI. The lilac-and-orchid visual identity avoids common hacker-style clichés while retaining a serious, modern security character.

3. Prompting Strategy

3.1 System Prompt Framework

The backend supplies a fixed system prompt before each user request. It defines the AI role, educational scope, safety boundary, response style, and redirection behavior. The prompt requires defensive guidance and refuses instructions that could enable credential theft, malware, unauthorized access, evasion, exploitation, or destructive activity.

3.2 Structured User Context

Each request is transformed into a structured prompt containing four fields: learning domain, learner level, response mode, and user question. This makes the model response more consistent than sending the raw question alone.

3.3 Sample Development Prompts

- Design a responsive security-awareness platform with a refined lilac theme, seven learning domains, and an accessible AI question composer.
- Create a FastAPI endpoint that validates a security-learning request and streams an external LLM response without exposing the API key to the browser.
- Refine the system prompt so unsafe offensive requests are refused and redirected toward defensive concepts or safe lab learning.
- Package the FastAPI application in a small Docker image that listens on port 8080 and runs as a non-root user.
- Review the mobile layout for touch targets, readability, wrapping, keyboard interaction, and reduced-motion accessibility.

3.4 Example Runtime Prompt

Example: Learning domain: Cloud Security | Learner level: beginner | Response mode: measures | User question: How can a small business prevent accidental exposure of cloud data?

4. Phase-by-Phase Development

1. Concept definition — selected a security-awareness platform aligned with academic and professional interests, then defined the target users and seven learning domains.
2. Experience design — compared visual directions and selected a dark glass interface, then replaced blue accents with a lilac, violet, and orchid palette.
3. Frontend development — built responsive navigation, topic cards, learning controls, question composer, streaming answer panel, copy action, and mobile breakpoints.

4. Backend development — created validated FastAPI routes, the AI system prompt, structured prompt construction, Groq streaming, safe error handling, and a health endpoint.
5. Containerization — created a Dockerfile, excluded secrets and unnecessary files, exposed port 8080, and configured a non-root runtime user.
6. Testing — compiled the Python and JavaScript, ran automated endpoint tests, and verified the homepage, health check, validation, and unconfigured-key behavior.
7. Cloud deployment — push the image to Amazon ECR, create an App Runner service, add the API key securely, verify HTTPS access, and insert the final URL into both documents.

5. Application Architecture

Item	Details
1. User interface	Captures the domain, level, mode, and question; displays streamed text without storing the secret key.
2. FastAPI server	Validates input, adds safety and learning instructions, and initiates the LLM request.
3. Groq LLM API	Generates the response and returns incremental content chunks.
4. Streaming path	The server forwards chunks to the browser, which appends them progressively to the answer panel.
5. AWS runtime	App Runner pulls the Docker image from ECR, injects environment variables, and exposes the HTTPS service.

6. Security and Responsible-AI Measures

- The Groq API key is read only from the server environment and is excluded from Git and Docker build context.
- Pydantic restricts input length and accepts only defined learning levels and response modes.
- The system prompt establishes a defensive-only boundary and safe redirection policy.
- Upstream error details and credentials are not returned to the browser.
- The Docker container runs as an unprivileged user.
- The application does not require user accounts or collect personal information for its core function.

7. Challenges and Resolutions

Item	Details
Broad initial concept	Converted a generic chatbot idea into defined learning domains and response modes.
Visual identity	Replaced the initial blue cyber palette with a distinctive lilac system while maintaining contrast.
Streaming	Used an asynchronous upstream stream and FastAPI StreamingResponse, then read chunks through the browser stream API.
Secret management	Kept the key in environment variables and added both .gitignore and .dockerignore safeguards.
Unsafe questions	Added a defensive system prompt that refuses harmful requests and redirects to safe concepts.
Deployment portability	Used Docker and a health endpoint so the same application can run locally and on AWS.

8. Testing and Validation

Automated tests verified the homepage response, health-check JSON, and rejection of an empty question. Python compilation and JavaScript syntax checks completed successfully. A local service smoke test confirmed HTTP 200 for the homepage, a healthy status response, and an appropriate configuration message when the API key was absent.

- Test result: 3 automated tests passed.
- Health endpoint: GET /health returned {status: healthy}.
- Security check: no API key appears in HTML, CSS, JavaScript, Dockerfile, or committed configuration.
- Final deployment checks pending: live LLM response, public AWS HTTPS URL, and mobile-browser review.

9. Key Learnings and Reflection

This project demonstrated that vibe coding is not simply asking AI to generate an application. Useful results depended on clear requirements, iterative prompting, evaluation of design

choices, validation of generated code, and understanding how each component communicates. The most important learning was connecting the visible user experience to backend security and cloud deployment decisions.

The project also reinforced that an AI feature needs boundaries. Because the application concerns security, defining a defensive educational purpose was as important as connecting the model. Containerization and the health endpoint showed how local application decisions affect the reliability of cloud deployment.

10. Final Deployment Checklist

- Create the Groq API key and test a live streamed response locally.
- Configure an AWS Budget alert before creating cloud resources.
- Build the Docker image and push it to Amazon ECR.
- Create the AWS App Runner service with port 8080 and /health monitoring.
- Add GROQ_API_KEY securely to the App Runner service configuration.
- Verify the application on desktop and mobile using its public HTTPS URL.
- Replace every [ADD AFTER DEPLOYMENT] marker with the final AWS URL.

Conclusion

CyberGuide AI satisfies the planned full-stack design: a clear and responsive frontend, secure FastAPI backend, external LLM integration with real-time streaming, Docker containerization, and an AWS deployment architecture. Once the API key and AWS service are configured, the final public URL will complete the required deliverables.